

第7章 函 数

7.1为什么要用函数

```
int main()
{
    {
        : 功能1内容
    }

    {
        : 功能2内容
    }

    {
        : 功能1内容
    }

    {
        : 功能2内容
    }
}
```

VS.

```
功能1函数()
{
    : 功能1内容
}

功能2函数()
{
    : 功能2内容
}

int main()
{
    调用功能1
    调用功能2
    调用功能1
    调用功能2
}
```

• 可读性、可维护性

如果程序的功能比较多，规模比较大，把所有代码都写在main函数中，就会使主函数变得庞杂、头绪不清，阅读和维护变得困难

• 代码重用率

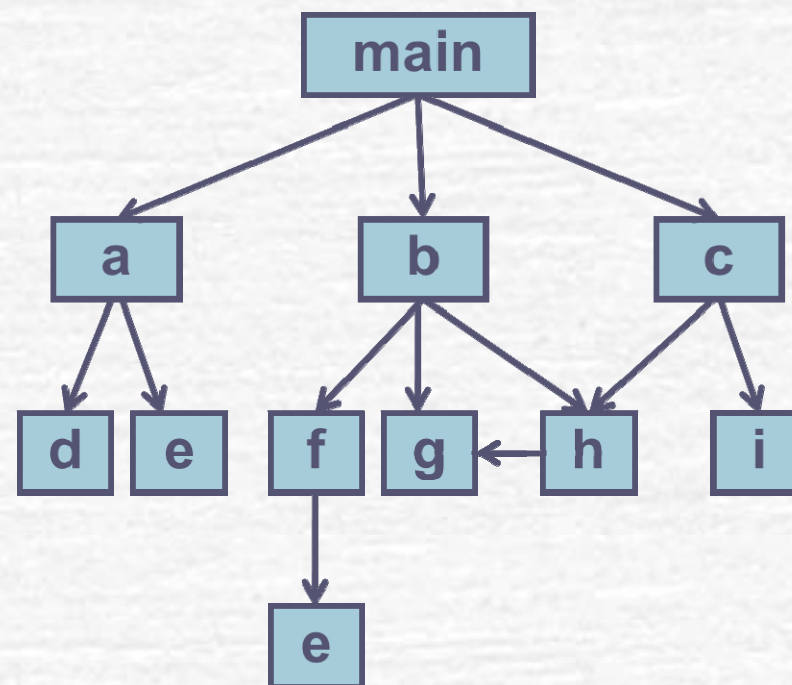
有时程序中要多次实现某一功能，就需要多次重复编写实现此功能的程序代码，这使程序冗长，不精炼

解决的方法：用模块化程序设计的思路

- 函数就是功能模块
- 每一个函数用来实现一个特定的功能
- 在设计一个较大的程序时，往往把它分为若干个程序模块，每一个模块包括一个或多个函数。



- ❏ C 程序可由一个主函数和若干个其他函数构成
- ❏ 主函数调用其他函数
- ❏ 其他函数也可以互相调用，但不能调用 `main()`；
- ❏ 同一个函数可以被一个或多个函数（包括自身）调用任意多次



结构化程序设计的基本思想

- ❖ 不要用**main()**来做所有的事情！
- ❖ 经验：一个函数的代码行最好不超出一个屏幕！
- ❖ 最好为程序要完成的每项任务编写单独的函数；
- ❖ **main()**除了调用其他函数之外不应该做太多事情；
- ❖ 每个被**main()**调用的函数完成任务后返回**main()**中发生调用的位置；
- ❖ **main()**继续执行直到程序结束，返回到操作系统。

C语言函数的组成

函数返回类型 函数名 函数参数

int f (int x)

↑ ↑ ↑

函数体

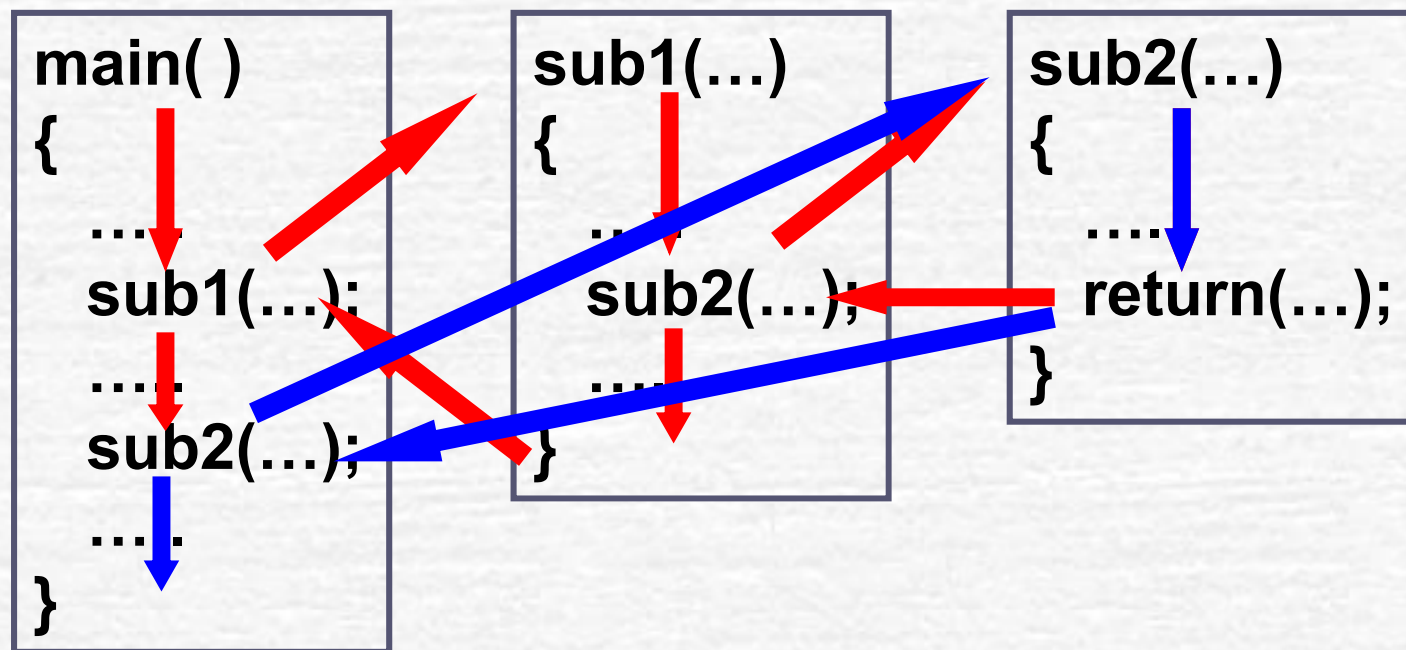
```
{  
    int r = 2*x+3;  
    return r;  
}
```


说明：

(1) 一个 **C 程序** 由一个或多个 **程序模块** 组成，每一个程序模块作为一个 **源程序文件**。

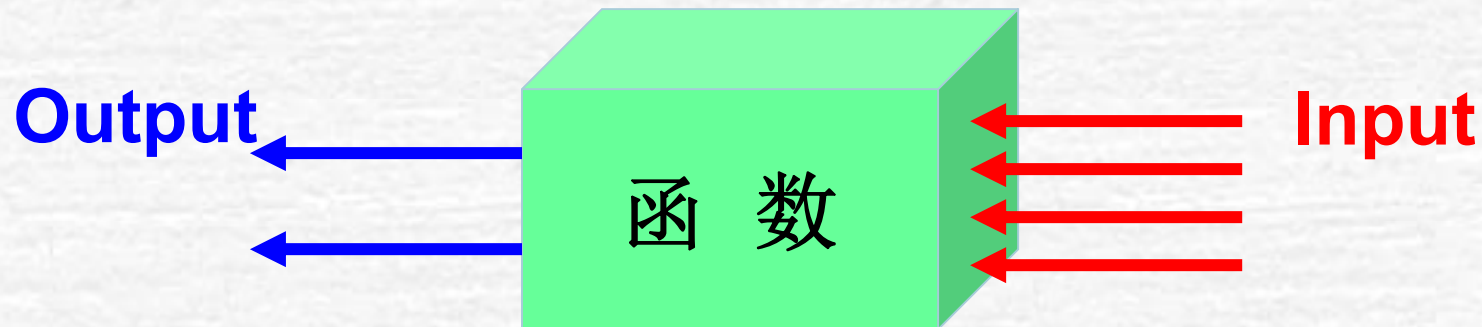
- 对较大的程序，一般不希望把所有内容全放在一个文件中，而是将它们分别放在若干个源文件中，由若干个源程序文件组成一个 **C** 程序。
- 一个 **源程序文件** 是一个 **编译单位**，在程序编译时是以源程序文件为单位进行编译的，而不是以函数为单位进行编译的。
- 这样便于分别编写、分别编译，提高调试效率。一个源程序文件可以为多个 **C** 程序共用。

(2) C程序的执行是从**main**函数开始的，如果在**main**函数中调用其他函数，在调用后流程返回到**main**函数，在**main**函数中结束整个程序的运行。



函数间的数据传递

- 函数的输入——函数中将要处理的数据对象，通常来自主调函数。
- 函数的输出——函数运行的最终结果，通常需返回给主调函数。



(3) 所有函数都是平行的，即在定义函数时是分别进行的，是互相独立的。

- 一个函数并不从属于另一个函数，即**函数不能嵌套定义**。
- **函数间可以互相调用**，但**不能调用main函数**。
main函数是被操作系统调用的。

(4) 从用户使用的角度看，函数有两种。

- **库函数**：它是由系统提供的，用户不必自己定义而直接使用它们。
 - 说明：不同的**C**语言编译系统提供的库函数的数量和功能会有一些不同，当然许多基本的函数是共同的。
- **用户自己定义的函数**：它是用以解决用户专门需要的函数。

(6) 从函数的形式看，函数分两类。

- ① **无参函数**：无参函数一般用来执行指定的一组操作。无参函数可以带回或不带回函数值，但一般以不带回函数值的居多。
- ② **有参函数**：在调用函数时，主调函数在调用被调用函数时，通过参数向被调用函数传递数据，一般情况下，执行被调用函数时会得到一个函数值，供主调函数使用。

```
int main(char argc, char *argv[])
```


7.2 怎样定义函数

7.2.1 为什么要定义函数

7.2.2 定义函数的方法

7.2.1 为什么要定义函数

- ✎ C语言要求，在程序中用到的所有函数，必须“先定义，后使用”
- ✎ 将下列信息通知编译系统：
 - 指定函数的名字，以便以后按名调用
 - 指定函数类型，即函数返回值的类型，以便传回结果
 - 指定函数参数的个数和类型，以便在调用函数时向它们传递数据
 - 指定函数的功能，这是最重要的，这是在函数体中解决的

7.2.2 定义函数的方法

1. 定义无参函数

指定函数值的类型

定义无参函数的形式为:

类型名 函数名()

{

函数体

}

类型名 函数名(void)

{

函数体

}

如: 例7.1中的

包括声明部分
和语句部分

message

2. 定义有参函数

定义有参函数的一般形式为:

类型名 函数名 (形式参数表列)

{

函数体

}

```
int max(int x,int y)
{ int z;           //声明部分
  z=x>y ? x : y;   //执行语句部分
  return(z);
}
```

3. 定义空函数

定义空函数的一般形式为：

类型名 函数名 ()
{ }

- 先用空函数占一个位置，以后逐步扩充
- 好处：程序结构清楚，可读性好，以后扩充新功能方便，对程序结构影响不大
- 函数也可以没有返回值，此时就用**void**来修饰返回类型标识符。

如：定义 `void dummy( ) { }`
调用 `dummy( );`

7.3 调用函数

7.3.1 函数调用的形式

7.3.2 函数调用时的数据传递

7.3.3 函数调用的过程

7.3.4 函数的返回值

7.3.1 函数调用的形式

- 函数调用的一般形式为：

函数名（实参表列）

- 如果是调用无参函数，则“实参表列”可以没有，但括号不能省略

如： **printstar(); printf_message();**

- 如果实参表列包含多个实参，则各参数间用逗号隔开

如： **c=max(a,b);**

- 按函数调用在程序中出现的形式和位置来分，可以有以下**3种函数调用方式**：

- 函数调用语句**

- 把函数调用单独作为一个语句

如**printf_star()**;

- 这时不要求函数带返回值，只要求函数完成一定的操作

- 如：**rewind, srand, clrscr**等；

- 按函数调用在程序中出现的形式和位置来分，可以有以下3种函数调用方式：

2. 函数表达式

- 函数调用出现在另一个表达式中

如： **c=max(a,b);**

- 这时要求函数带回一个确定的值以参加表达式的运算

如： **c=getchar (); c=2*max(a,b);**

- 按函数调用在程序中出现的形式和位置来分，可以有以下3种函数调用方式：

3. 函数参数

- 函数调用作为另一函数调用时的实参

如 **`m=max(a,max(b,c));`**

- 其中 `max(b,c)` 是一次函数调用，它的值作为 `max` 另一次调用的实参

- 如：
`m=max(a,max(b,c));`
`printf("%d",max(a,b));`

7.3.2 函数调用时的数据传递

1. 形式参数和实际参数

- 在调用有参函数时，**主调函数**和**被调用函数**之间有数据传递关系
- 定义函数时函数名后面的变量名称为“**形式参数**”（简称“**形参**”）
- 主调函数中调用一个函数时，函数名后面参数称为“**实际参数**”（简称“**实参**”）
 - **实际参数可以是常量、变量或表达式**

2. 实参和形参间的数据传递

- 在调用函数过程中，系统会把实参的值传递给被调用函数的形参
- 或者说，形参从实参得到一个值
- 该值在函数调用期间有效，可以参加被调函数中的运算

例7.2 输入两个整数，要求输出其中值较大者。要求用函数来找到大数。

☞ 解题思路：

(1)函数名应是见名知意，今定名为max

(2) 由于给定的两个数是整数，返回主调函数的值（即较大数）应该是整型

(3)max函数应当有两个参数，以便从主函数接收两个整数，因此参数的类型应当是整型

先编写max函数：

```
int max(int x,int y)  
{  
    int z;  
    z=x>y?x:y;  
    return(z);  
}
```

在**max**函数上面，再编写主函数

```
#include <stdio.h>
```

```
int main()
```

```
{ int max(int x,int y);  int a,b,c;
```

```
    printf("two integer numbers: ");
```

```
    scanf("%d,%d",&a,&b);
```

```
    c=max[a,b]; 实参可以是常量、变量或表达式
```

```
    printf("max is %d\n",c);
```

```
}
```

```
two integer numbers:12,-34
max is 12
```


c=max(a,b);

(**main**函数)

int max(int x, int y)

(**max**函数)

{

int z;

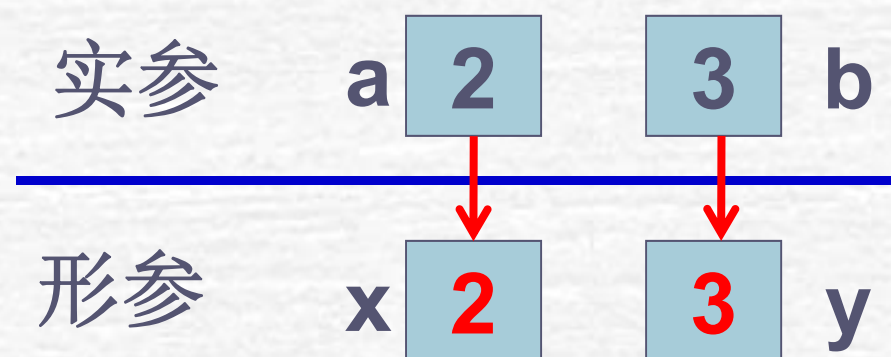
z=x>y?x:y;

return(z);

}

7.3.3 函数调用的过程

在定义函数中指定的形参，在未出现函数调用时，它们并不占内存中的存储单元。
在发生函数调用时，函数**max**的形参被临时分配内存单元。



- 被调函数中，形参可以参与运算
- 调用结束，形参单元被释放
- 实参单元仍保留并维持原值，没有改变
- 如果在执行一个被调用函数时，形参的值发生改变，不会改变主调函数的实参的值

实参 a **2** **3** b

7.3.4. 函数的返回值

➤通常，希望调用函数能得到一个确定的返回值)

```
max(int x,int y)
```

```
{
```

```
    return(x>y? x:y);
```

```
}
```

调函数能得
(函数的返

(1) 函数的返回值是通过函数中的**return**语句获得的。

◆一个函数中可以有一个以上的**return**语句，执行到哪一个**return**语句，哪一个就起作用

◆**return**语句后面的括号可以不要

return 语句的一般形式:

return [() 表达式 []] ;

或 **return ;**

return不是必须的; 当函数体执行完毕后, 若未执行过**return**语句, 则并不表示函数不带返回值, 而是其带回的值是不确定的。

一个函数中可以有多个**return**语句, 先执行到的那个**return**语句起作用。

➤通常，希望通过函数调用使主调函数能得到一个确定的值，这就是函数值(函数的返回值)

(2) 函数值的类型。应当在定义函数时指定函数值的类型

```
int max (float x,float y)
char letter (char c1,char c2)
double min (int x,int y)
```

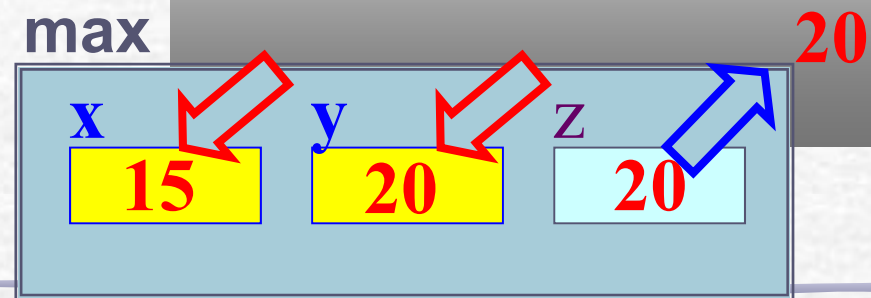

【例7.2】

```
#include <stdio.h>
```

```
→ void main()  
→ { int max(int x, int y);  
→   int a,b,c;  
→   scanf("%d,%d",&a,&b);  
→   c=max(a,b);  
→   printf("Max is %d",c);  
→ }  
→ int max(int x, int y)  
→ { int z;  
→   z=x>y?x:y;  
→   return(z);  
→ }
```

```
15, 20 ✓  
Max is 20
```

函数定义中的形参在未执行到函数调用时并不占内存中的存储单元。只有发生函数调用时，才给该函数的形参分配空间，且当该函数调用结束时，形参所占的内存单元被释放。



➤通常，希望通过函数调用使主调函数能得到一个确定的值，这就是函数值(函数的返回值)

(3)在定义函数时指定的函数类型一般应该和return语句中的表达式类型一致

◆如果函数值的类型和return语句中表达式的值不一致，则以函数类型为准

例7.3将例7.2稍作改动，将在**max**函数中定义的变量**z**改为**float**型。函数返回值的类型与指定的函数类型不同，分析其处理方法。

➤解题思路：如果函数返回值的类型与指定的函数类型不同，按照赋值规则处理。


```
#include <stdio.h>
int main()
{ int max(float x,float y);
  float a,b; int c;
  scanf("%f,%f",&a,&b);
  c=max(a,b);
  printf("max is %d\n",c);
  return 0;
}
```

```
1.5,2.6
max is 2
```

```
int max(float x,float y)
{ float z;
  z=x>y?x:y;
  return( z );
}
```

1.5 2.6



```
2.6
```



2

➤通常，希望通过函数调用使主调函数能得到一个确定的值，这就是函数值(函数的返回值)

(4)对于不带回值的函数，应当定义函数为“**void**类型”（或称空类型）。

◆ 此时，禁止在调用函数中使用被调用函数的返回值，并且在被调用函数的函数体内不得出现**return <表达式>**语句（只能用**return;**）。

◆ 如：例7.1中不允许形如“**a=printstar()**”的调用。

7.4对被调用函数的声明和函数原型

在一个函数中调用另一个函数需要具备如下条件：

- (1) 被调用函数必须是已经定义的函数（是库函数或用户自己定义的函数）
- (2) 如果使用库函数，应该在本文件开头加相应的 `#include` 指令
- (3) 如果使用自己定义的函数，而该函数的位置在调用它的函数后面，应该**声明**

```
#include <stdio.h>
#define PI 3.1415926
int main()
{    //主函数
    float radius, a;
    radius = 10.0;
    a=circle_area(radius); //调用函数1
    show_area(a);          //调用函数2
    return 0;
}
float circle_area(float r)
{    // 函数1：计算圆面积的函数
    float tmp = PI*r*r;
    return tmp;
}
void show_area(float a)
{    // 函数2：输出圆面积的函数
    printf("Area = %f\n", a);
}
```

这个程序有何问题？

函数的定义与函数的声明的区别：

- **函数的定义** —— 指的是对函数功能的确立，包括指定函数名，函数值类型，形参及其类型，函数体等。
- **函数的声明**（**C**语言中称为**函数原型**）—— 其作用是把函数的名字，函数类型以及形参的类型、个数和顺序通知编译系统，以便在编译时以此为依据作对照检查，及时发现并纠正错误（语法错误）。

7.4对被调用函数的声明和函数原型

例7.4 输入两个实数，用一个函数求出它们之和。

解题思路：

- 用add函数实现。首先要定义add函数，它为float型，它应有两个参数，也应为float型。特别要注意的是：要对add函数进行声明。
- 分别编写add函数和main函数，它们组成一个源程序文件
- main函数的位置在add函数之前
- 在main函数中对add函数进行声明

```
#include <stdio.h>
```

```
int main()
```

```
{ float add(float x, float y);
```

```
float a,b,c;
```

```
printf("Please enter a and b:");
```

```
scanf("%f,%f",&a,&b);
```

```
c=add(a,b);
```

```
printf("sum is %f\n",c);
```

```
return 0;
```

```
}
```

对**add**函数声明

```
float add(float x,float y)
```

```
{ float z;
```

```
z=x+y;
```

```
return(z);
```

求两个实数之和，
函数值也是实型


```
#include <stdio.h>
```

```
int main()
```

```
{ float add(float x, float y);
```

```
float a,b,c;
```

```
printf("Please enter a and b:");
```

```
scanf("%f,%f",&a,&b);
```

```
c=add(a,b);
```

```
printf("sum is %f\n",c);
```

```
return 0;
```

```
}
```

只差一个分号

```
float add(float x,float y)
```

```
{ float z;
```

```
z=x+y;
```

```
return(z);
```

```
}
```

```
#include <
int main()
```

```
{ float add(float x, float y);
  float a,b,c;
  printf("Please enter a and b:");
  scanf("%f,%f",&a,&b);
  c=add(a,b);
  printf("sum is %f\n",c);
  return 0;
}
```

定义**add**函数

```
float add(float x,float y)
{ float z;
  z=x+y;
  return(z);
}
```

调用**add**函数

```
Please enter a and b:3.6,6.5
sum is 10.100000
```

函数原型的一般形式是：

1. 函数类型 函数名(参数类型1 参数名1,
参数类型2 参数名2);

如: **float add(float a, float b);**

2. 函数类型 函数名(参数类型1, 参数类型2.....);

如: **float add(float, float);**

应当确保函数原型与函数首部写法一致！

- 函数声明中可以不给出形参名，因为编译系统不检查参数名；但形参的数据类型不可省略！

- 如果在函数调用之前，没有对该函数作声明，则编译系统会把第一次遇到的该函数形式（函数定义/函数调用）作为函数的声明，且若遇到的是函数调用将把函数类型默认为**int**型。
- 如果被调用函数的定义出现在主调函数之前，可以不必再加以声明。
- 如果在所有函数定义前，且在函数的外部，已做了函数声明，则在各个主调函数中不必对所调用函数再作声明。（头文件中大体如此。）

```
char letter(char, char);           //以下 3 行在所有函数之前,且在函数外部
float f(float, float);
int i (float, float);

int main()                          //在 main 函数中要调用 letter, f 和 i 函数
{
    :
}
//下面定义被 main 函数调用的 3 个函数

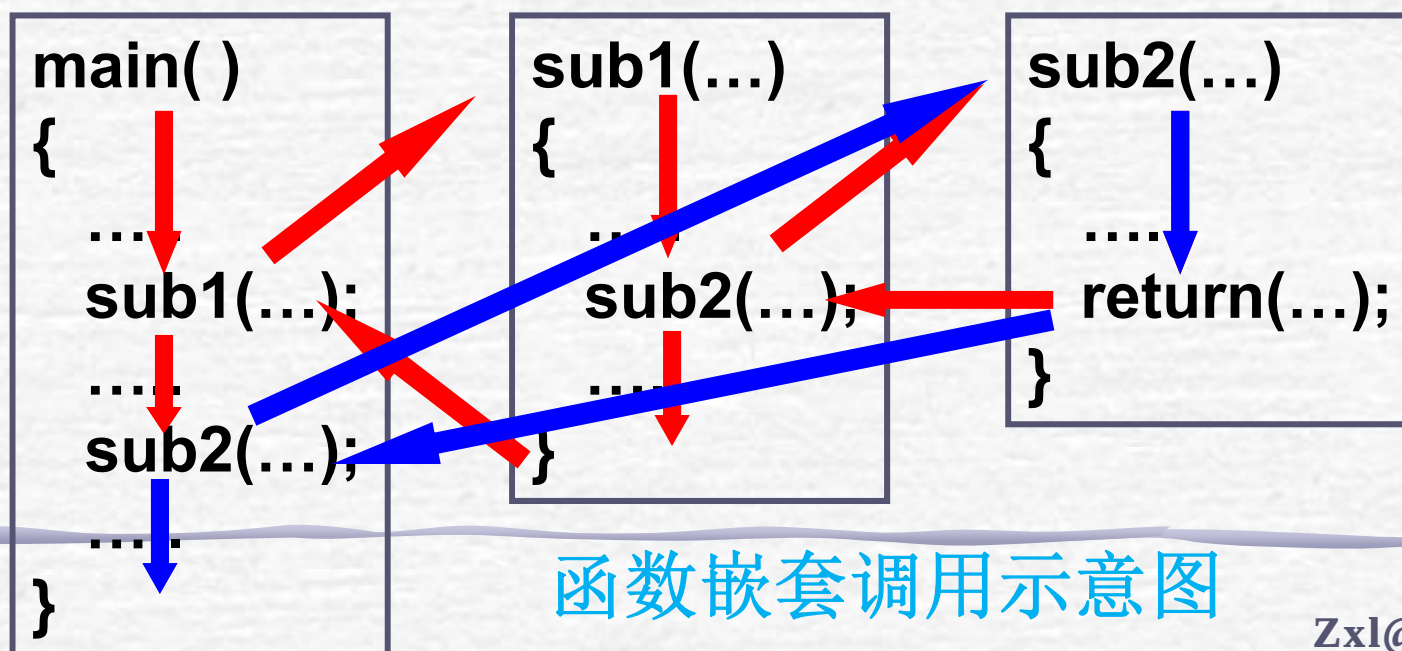
char letter(char c1, char c2)      //定义 letter 函数
{
    :
}

float f(float x, float y)          //定义 f 函数
{
    :
}

int i(float j, float k)            //定义 i 函数
{
    :
}
```

7.5 函数的嵌套调用

- ❖ **C语言**的函数定义都是互相平行、独立的，也就是说在定义函数时，一个函数内不能包含另一个函数的定义，即**C语言不允许嵌套定义函数**。
- ❖ **C语言允许嵌套调用函数**，即在调用一个函数的过程中，还可以调用另一函数（甚至该函数子自身，称为**递归调用**）。



函数嵌套调用示意图

7.5 函数的嵌套调用

例7.5 输入4个整数，找出其中最大的数。
用函数的嵌套调用来处理。

解题思路：

- `main`中调用`max4`函数，找4个数中最大者
- `max4`中再调用`max2`，找两个数中的大者
- `max4`中多次调用`max2`，可找4个数中的大者，然后把它作为函数值返回`main`函数
- `main`函数中输出结果

主函数

```
#include <stdio.h>
```

```
int main()
```

```
{ int max4(int a,int b,int c,int d);
```

```
    int a,b,c,d,max;
```

```
    printf("4 interger numbers:");
```

```
    scanf("%d%d%d%d",&a,&b,&c,&d);
```

```
    max=max4(a,b,c,d);
```

```
    printf("max=%d \n",max);
```

```
    return 0;
```

```
}
```

对**max4** 函数声明

输入**4**个整数

调用后肯定是**4**
个数中最大者

输出最大者

max4函数

```
int max4(int a,int b,int c,int d)
```

```
{ int max2(int a,int b);
```

对max2 函数声明

```
int m;
```

```
m=max2(a,b);
```

a,b中较大者

```
m=max2(m,c);
```

a,b,c中较大者

```
m=max2(m,d);
```

a,b,c,d中最大者

```
return(m);
```

```
}
```


max4函数

```
int max4(int a,int b,int c,int d)
{  int max2(int a,int b);
    int m;
    m=max2(a,b);
    m=max2(m,c);
    m=max2(m,d);
    return(m);
}
```

找a,b中较大者

max2函数

```
int max2(int a,int b)
{  if(a>=b)
    return a;
    else
    return b;
}
```

return(a>=b?a:b);

max4函数

```
int max4(int a,int b,int c,int d)
```

```
{ int max2(int a,int b);
```

```
  int m;
```

```
  m=max2(a,b);
```

```
  m=max2(m,c);
```

```
  m=max2(m,d);
```

```
  return(m);
```

```
}
```

m=max2(max2(a,b),c);

```
int max2(int a,int b)
```

```
{ return(a>=b?a:b); }
```

max4函数

```
int max4(int a,int b,int c,int d)
```

```
{ int max2(int
```

```
m=max2(max2(max2(a,b),c),d);
```

```
int m;
```

```
m=max2(a,b);
```

```
m=max2(m,c);
```

```
m=max2(m,d);
```

```
return(m);
```

```
}
```

```
int max2(int a,int b)
```

```
{ return(a>b?a:b); }
```


max4函数

```
int max4(  
{ int max2(int a,  
    int m;  
    m=max2(a,b);  
    m=max2(m,c);  
    m=max2(m,d);  
    return(m);  
}
```

```
int max2(int a,int b)  
{ return(a>b?a:b); }
```

```
#include
```

```
int main()
```

```
{ .....
```

```
max=max4(a,b,c,d);
```

```
.....
```

```
}
```

```
int max4(int a,int b,int c,int
```

```
{ int max2(int a,int b);
```

```
return max2(max2(max2(a,b),c),d);
```

```
}
```

```
int max2(int a,int b)
```

```
{ return(a>b?a:b); }
```

```
4 interger numbers:12 45 -6 89
max=89
```

```
#include <stdio.h>
```

```
int max4(int a,int b,int c,int d);
```

```
int max2(int a,int b);
```

```
int main()
```

```
{ int a,b,c,d,max;
```

```
printf("4 interger numbers:");
```

```
scanf("%d%d%d%d",&a,&b,&c,&d);
```

```
max=max4(a,b,c,d);
```

```
printf("max=%d \n",max);
```

```
return 0;
```

```
}
```

```
int max4(int a,int b,int c,int d)
```

```
{ return max2(max2(max2(a,b),c),d); }
```

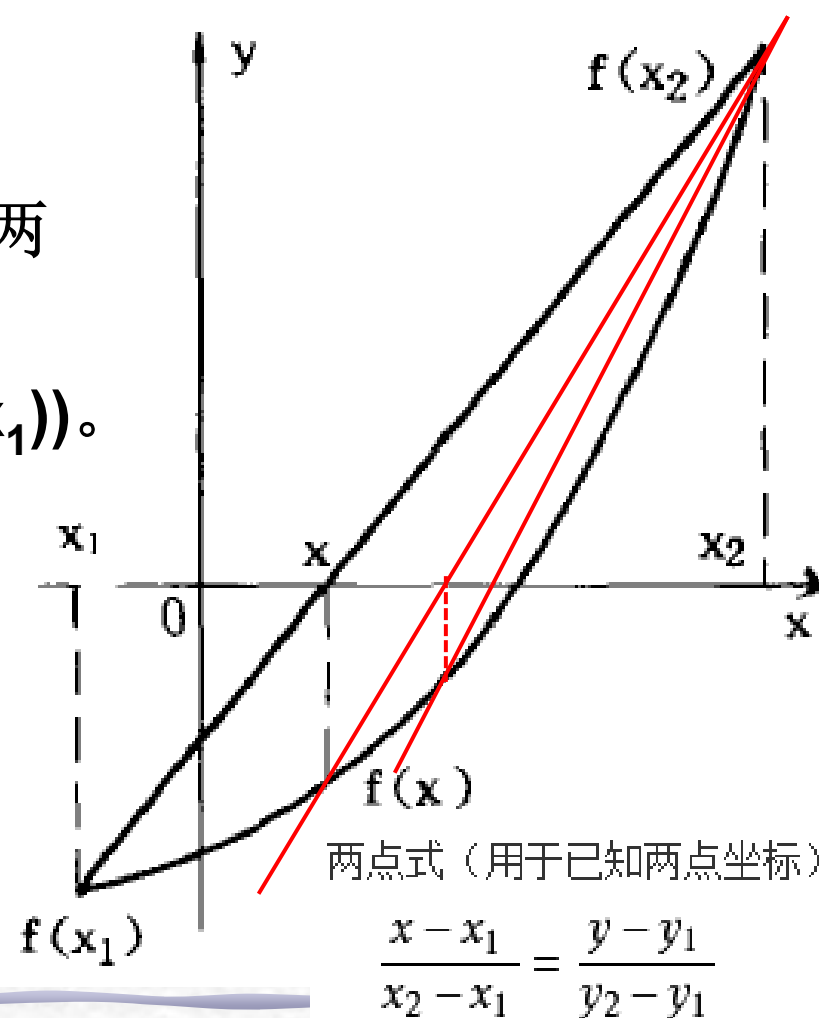
```
int max2(int a,int b)
```

```
{ return(a>b?a:b); }
```

【例Ex8.6】 用弦截法求方程 $x^3-5x^2+16x-80=0$ 的根。

弦截法

- 1、取 $x_1 \neq x_2$ ，且 $f(x_1)f(x_2) < 0$ 。
- 2、连接 $(x_1, f(x_1))$ 和 $(x_2, f(x_2))$ 两点，与 x 轴交于点 $(x, 0)$ ，其中 $x = (x_1 f(x_2) - x_2 f(x_1)) / (f(x_2) - f(x_1))$ 。
- 3、求 $f(x)$ ，若 $|f(x)| < \varepsilon$ ，则认为 $f(x) \approx 0$ ，算法结束。
- 4、否则，若 $f(x)f(x_1) > 0$ ，则 $x_1 = x$ ；否则 $x_2 = x$ 。



【例Ex8.6】用弦截法求方程 $x^3-5x^2+16x-80=0$ 的根。

弦截法

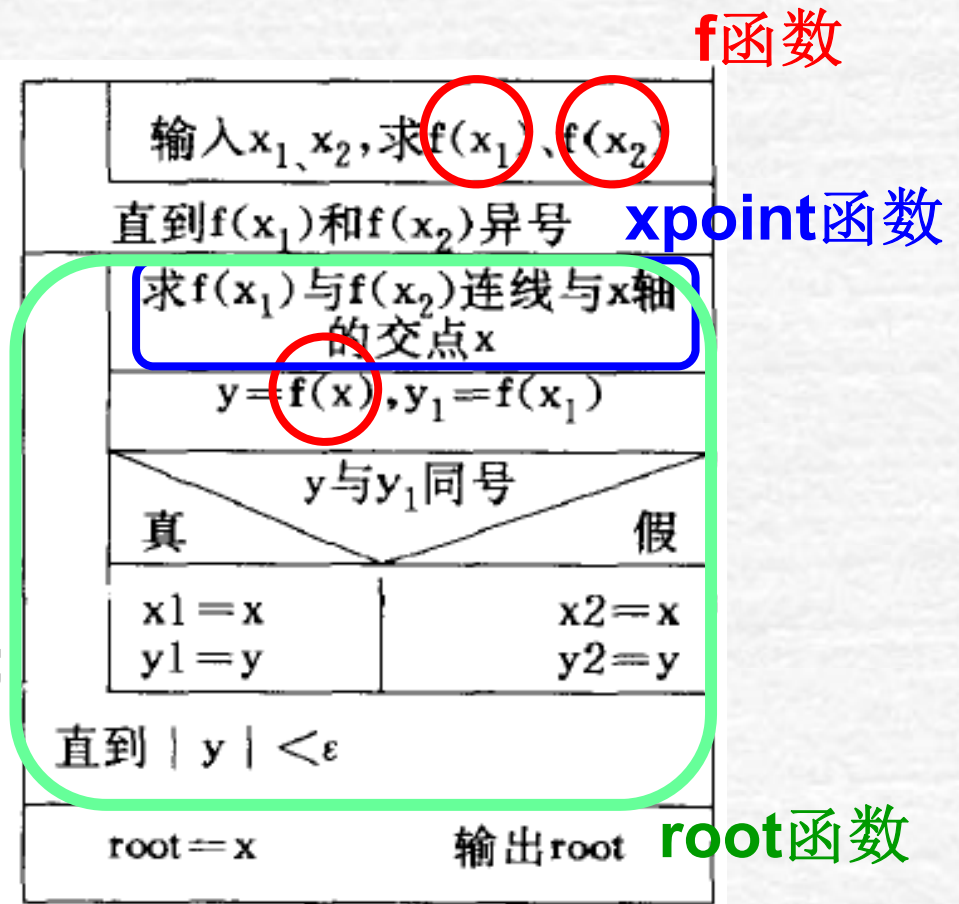
程序N-S流程图

$|f(x)| < \varepsilon$ 的表示

1、 $f(x) < \varepsilon \&\& f(x) > -\varepsilon$

2、用求绝对值函数：

```
#include <math.h>
double fabs (double x);
int abs (int x);
```



【例Ex8.6】 用弦截法求方程 $x^3-5x^2+16x-80=0$ 的根。

```
# include <stdio.h>
```

```
# include <math.h>
```

```
/*求函数f (x)的值*/
```

```
float f ( float x ) {
```

```
    float y=((x-5.0)*x+16.0)*x-80.0;
```

```
    return(y);
```

```
}
```

```
/*求弦与x轴交点的x坐标*/
```

```
float xpoint ( float x1, float x2 ) {
```

```
    float y=(x1*f(x2)-x2*f(x1))/(f(x2)-f(x1));
```

```
    return(y);
```

```
}
```

/*求近似根*/

float root (float x1, float x2) {

int i;

float x,y,y1;

y1=f(x1);

do {

x=xpoint(x1,x2);

y=f(x);

if (y*y1>0) {

x1=x; y1=y;

}

else {

x2=x; y2=y;

}

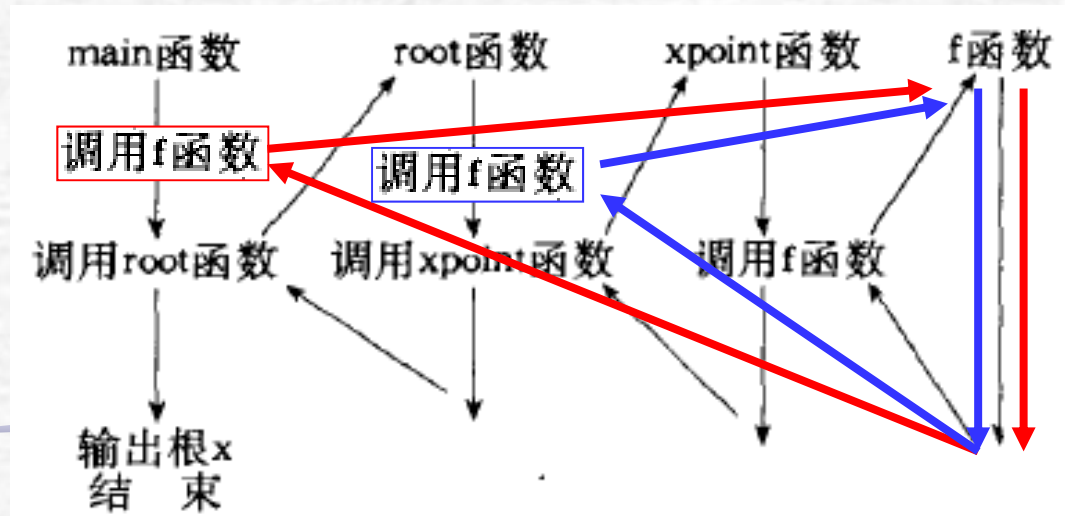
} while (fabs(y)>0.0001);

return (x);

}


```
main( ) {
    float x1,x2,f1,f2,x;
    do{
        printf("input x1,x2:\n");
        scanf("%f,%f",&x1,&x2);
        f1=f(x1); f2=f(x2);
    } while ( f1*f2>=0 );
    x=root( x1,x2 );
    printf("A root of equation is %8.4f\n",x);
}
```

函数调用关系图



7.7 数组作为函数参数

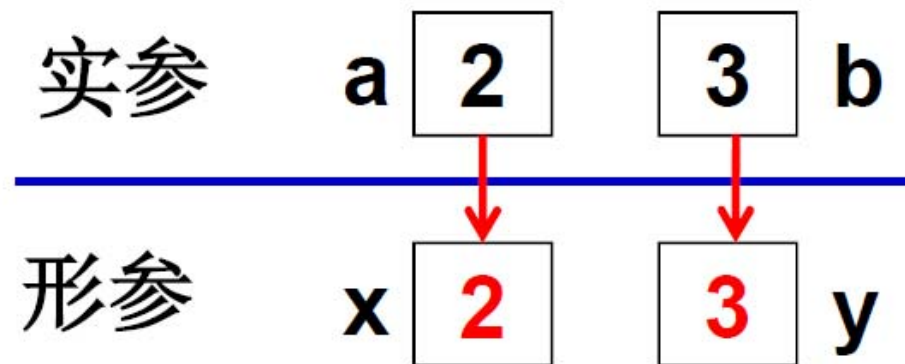
7.7.1 数组元素作函数实参

7.7.2 数组名作函数参数

7.7.3 多维数组名作函数参数

回顾：函数调用过程

- **调用结束**，形参单元被释放，实参单元仍保留并维持原值，没有改变
- 如果在执行一个被调用函数时，**形参的值发生改变**，不会改变主调函数的实参的值



7.7.1 数组元素作函数实参

【例7.9】 输入10个数，要求输出其中值最大的元素和该数是第几个数。

解题思路：

- 定义数组a，用来存放10个数
- 设计函数max，用来求两个数中的大者
- 在主函数中定义变量m，初值为a[0]，每次调用max函数后的返回值存放在m中
- 用“打擂台”算法，依次将数组元素a[1]到a[9]与m比较，最后得到的m值就是10个数中的最大者

```
#include <stdio.h>
```

```
int main()
```

```
{ int max(int x,int y);
```

```
    int a[10],m,n,i;
```

```
    printf("10 integer numbers:\n");
```

```
    for(i=0;i<10;i++)
```

```
        scanf("%d",&a[i]);
```

```
    printf("\n");
```

```
10 integer numbers:
```

```
4 7 0 -3 4 34 67 -42 31 -76
```

```
for(i=1,m=a[0],n=0;i<10;i++)
{ if (max(m, a[i])>m)
    { m=max(m, a[i]);
      n=i;
    }
}
printf("largest number is %d\n",m);
printf("%dth number.\n",n+1);
}
```

largest number is 67
7th number.

```
int max(int x, int y)
{ return (x>y? x: y); }
```


7.7.2数组名作函数参数

- 除了可以用数组元素作为函数参数外，还可以用数组名作函数参数(包括实参和形参)
- 用数组元素作实参时，向形参变量传递的是数组元素的值
- 用数组名作函数实参时，向形参传递的是数组首元素的地址

7.7.2数组名作函数参数

例7.10 有一个一维数组score，内放10个学生成绩，求平均成绩。

解题思路：

- 用函数average求平均成绩，用数组名作为函数实参，形参也用数组名
- 在average函数中引用各数组元素，求平均成绩并返回main函数

```
#include <stdio.h>
int main()
{ float average(float array[10]);
  float score[10],aver; int i;
  printf("input 10 scores:\n");
  for(i=0;i<10;i++)
    scanf("%f",&score[i]);
  printf("\n");
  aver=average(score);
  printf("%5.2f\n",aver);
  return 0;
}
```

定义实参数组

定义形参数组

```
float average(float array[10])
```

```
{ int i;
```

```
    float aver,sum=array[0];
```

```
    for(i=1;i<10;i++)
```

```
        sum=sum+array[i];
```

```
    aver=sum/10;
```

```
    return(aver);
```

```
}
```

相当于score[0]

相当于score[i]

```
input 10 scores:
100 56 78 98 67.5 99 54 88.5 76 58

77.50
```

【例7.11】 有两个班级，分别有35名和30名学生，调用一个**average**函数，分别求这两个班的学生们的平均成绩。

解题思路：

- 需要解决怎样用同一个函数求两个不同长度的数组的平均值的问题
- 定义**average**函数时不指定数组的长度，在形参表中增加一个整型变量*i*
- 从主函数把数组实际长度从实参传递给形参*i*
- 这个*i*用来在**average**函数中控制循环的次数
- 为简化，设两个班的学生数分别为5和10

```
#include <stdio.h>
int main()
{ float average(float array[ ], int n);
  float score1[5]={98.5,97,91.5,60,55};
  float score2[10]={67.5,89.5,99,69.5,
                    77,89.5,76.5,54,60,99.5};
  printf("%6.2f\n",average(score1,5));
  printf("%6.2f\n",average(score2,10));
  return 0;
}
```


调用形式为**average(score1, 5)**时

```
float average(float array[ ],int n)
```

```
{ int i;
```

相当于**5**

```
float aver, sum=array[0];
```

```
for(i=1;i<n;i++)
```

相当于**score1[0]**

```
sum=sum+array[i];
```

```
aver=sum/n;
```

相当于**score1[i]**

```
return(aver);
```

```
}
```

调用形式为**average(score2, 10)**时

```
float average(float array[ ],int n)
```

```
{ int i;
```

相当于10

```
float aver, sum=array[0];
```

```
for(i=1;i<n;i++)
```

相当于score2[0]

```
sum=sum+array[i];
```

```
aver=sum/n;
```

相当于score2[i]

```
return(aver);
```

```
}
```

80.40

78.20

【例7.12】用**选择法**对数组中**10**个整数按由小到大排序。

解题思路：

- 所谓选择法就是先将**10**个数中最小的数与**a[0]**对换；再将**a[1]**到**a[9]**中最小的数与**a[1]**对换.....每比较一轮，找出一个未经排序的数中最小的一个
- 共比较**9**轮

a[0] a[1] a[2] a[3] a[4]



小到大排序

```
#include <stdio.h>

int main()
{ void sort(int array[],int n);
  int a[10],i;
  printf("enter array:\n");
  for(i=0;i<10;i++) scanf("%d",&a[i]);
  sort(a,10);
  printf("The sorted array:\n");
  for(i=0;i<10;i++) printf("%d ",a[i]);
  printf("\n");
  return 0;
}
```

```
void sort(int array[],int n)
```

实参数组与形参数组类型应该一致；如果不一致，会有警告，而且结果将出错！（指针类型不同）

```
for(i=0;i<n-1;i++)
```

```
{ k=i;
```

最小数与sort[i]对换

```
for(j=i+1;j<n;j++)
```

```
if(array[j]<array[k]) k=j;
```

```
t=array[k]; if (k==i) continue;
```

```
array[k]=array[i];
```

```
array[i]=t; enter array:
```

```
}
```

```
}
```

由于传递的是地址，所以形参数组和实参数组共享同一段存储空间。因此，函数中对形参数组元素值的修改，同时也就修改了实参数组的对应元素值！

为什么能在sort函数内
改变数组a的内容？

7.7.3 多维数组名作函数参数

例7.13 有一个 3×4 的矩阵，求所有元素中的最大值。

- 解题思路：先使变量`max`的初值等于矩阵中第一个元素的值，然后将矩阵中各个元素的值与`max`相比，每次比较后都把“大者”存放在`max`中，全部元素比较完后，`max` 的值就是所有元素的最大值。

```
#include  
int main()
```

第一维的长度
可以省略

第二维长度不能省略
且要与形参数组
第二维大小相同

```
{ int max_value(int array[][4]);  
  int a[3][4]={1,3,5,7},{2,4,6,8},  
              {15,17,34,12}};  
  printf("Max value is %d\n",  
         max_value(a));  
  return 0;  
}
```

- 除第一维大小可省略外，应说明其它各维的大小
【原因】数组元素在内存中是按行存放的，并不区分行列，
如不指定其各维的大小无法确定数组的规格。

要与实参数组第二维大小相同

```
int max_value(int array[][4])  
{ int i,j,max;  
  max = array[0][0];  
  for (i=0;i<3;i++)  
    for(j=0;j<4;j++)  
      if (array[i][j]>max)  
        max = array[i][j];  
  return (max);  
}
```

 ,int n

7.8局部变量和全局变量

7.8.1 局部变量

7.8.2 全局变量

变量

变量(variable) ——内存中特定的存储单元，通常用来存放在编写程序时值未知/不能确定的数据。

- 在程序运行过程中，其值可以改变的量。
- 通常用来保存输入数据、计算的中间结果和最终结果
- 每个变量占用若干个连续的字节，所占用的字节数由其数据类型

- 通常

何时分配？何时释放？——生存期

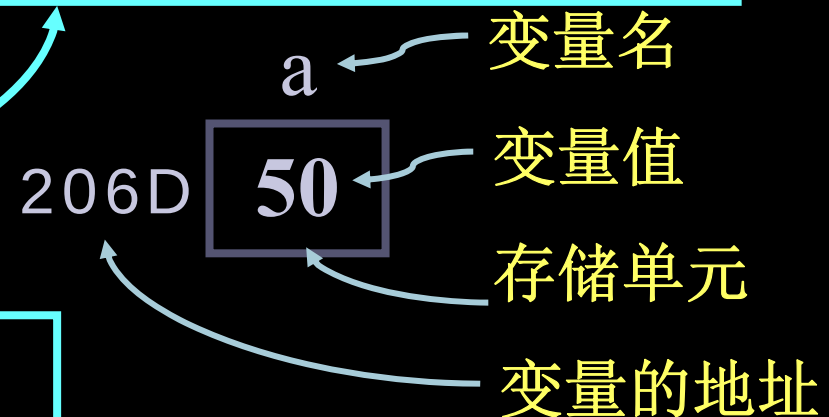
分配在何处？——存储方式

如何存储数据/编码方案？——数据类型

变量三要素

1. **变量名** —— 符号地址
2. **内存中的存储空间**
3. **变量值** —— 存放的值

程序中哪些模块内可访问这些变量？——作用域



变量的分类

89

1. 按数据类型分：整型、实型、字符型等等；
2. 从变量的作用域分：
 - 全局/外部变量、局部变量；
3. 从变量的存储方式、生存期分：
 - 静态、动态两大类；
 - 具体包括自动（**auto**）、静态（**static**）、寄存器（**register**）和外部（**extern**）四种；

7.8.1 局部变量

- 定义变量可能有三种情况：
 - 在函数的开头定义
 - 在函数内的复合语句内定义
 - 在函数的外部定义

7.8.1 局部变量

- 在一个函数内部定义的变量（包括形参）只在本函数范围内有效
- 在复合语句内定义的变量只在本复合语句范围内有效
- 在函数内部或复合语句内部定义的变量称为“**局部变量**”

```
float f1( int a)
{ int b,c;
  .....
}
```

a、b、c仅在此函数内有效

```
char f2(int x,int y)
{ int i,j;
  .....
}
```

x、y、i、j仅在此函数内有效

```
int main( )
{ int m,n;
  .....
  return 0;
}
```

m、n仅在此函数内有效


```
float f1( int a)
```

```
{ int b,c;
```

```
.....
```

```
}
```

```
char f2(int x,int y)
```

```
{ int i,j;
```

```
.....
```

```
}
```

```
int main( )
```

```
{ int a,b;
```

```
.....
```

```
return 0;
```

```
}
```

类似于不同
班同名学生

a、b也仅在此
函数内有效

```
int main ( )  
{ int a,b;
```

.....

```
{ int c;  
  c=a+b;
```

.....

```
}
```

.....

```
}
```

a、b仅在此复合语句内有效

c仅在此复合语句内有效

7.8.2全局变量

- 在函数内定义的变量是局部变量，而在函数之外定义的变量称为**外部变量**
- 外部变量是**全局变量**(也称全程变量)
- 全局变量可以为本文件中其他函数所共用
- 有效范围为从定义变量的位置开始到本源文件结束


```
int p=1,q=5
```

```
float f1(int a)
```

```
{ int b,c; ..... }
```

```
char c1,c2;
```

```
char f2 (int x, int y)
```

```
{ int i,j; ..... }
```

```
int main ( )
```

```
{ int m,n;
```

```
.....
```

```
return 0;
```

```
}
```

p、q、c1、c2
为全局变量

```
int p=1,q=5  
float f1(int a)  
{ int b,c; ..... }
```

p、q的有效范围

```
char c1,c2;  
char f2 (int x, int y)  
{ int i,j; ..... }  
int main ( )  
{ int m,n;  
    .....  
    return 0;  
}
```

c1、c2的有效范围

7.8.2 全局变量

- 使用全局变量的优点：增加了函数间数据联系的渠道，减少函数实参与形参的个数，从而减少内存空间以及传递数据时的时间消耗。
 - 若在一个函数中改变了全局变量的值，则就能影响到其它函数。
- 使用全局变量的缺点：
 - 全局变量在全部执行过程中都占用存储单元。
 - 使用全局变量将降低函数的通用性和可靠性（依赖于其定义所处的环境）。
 - 使用全局变量过多，将导致程序的清晰性降低。

因此，不到必要时尽量避免使用全局变量！

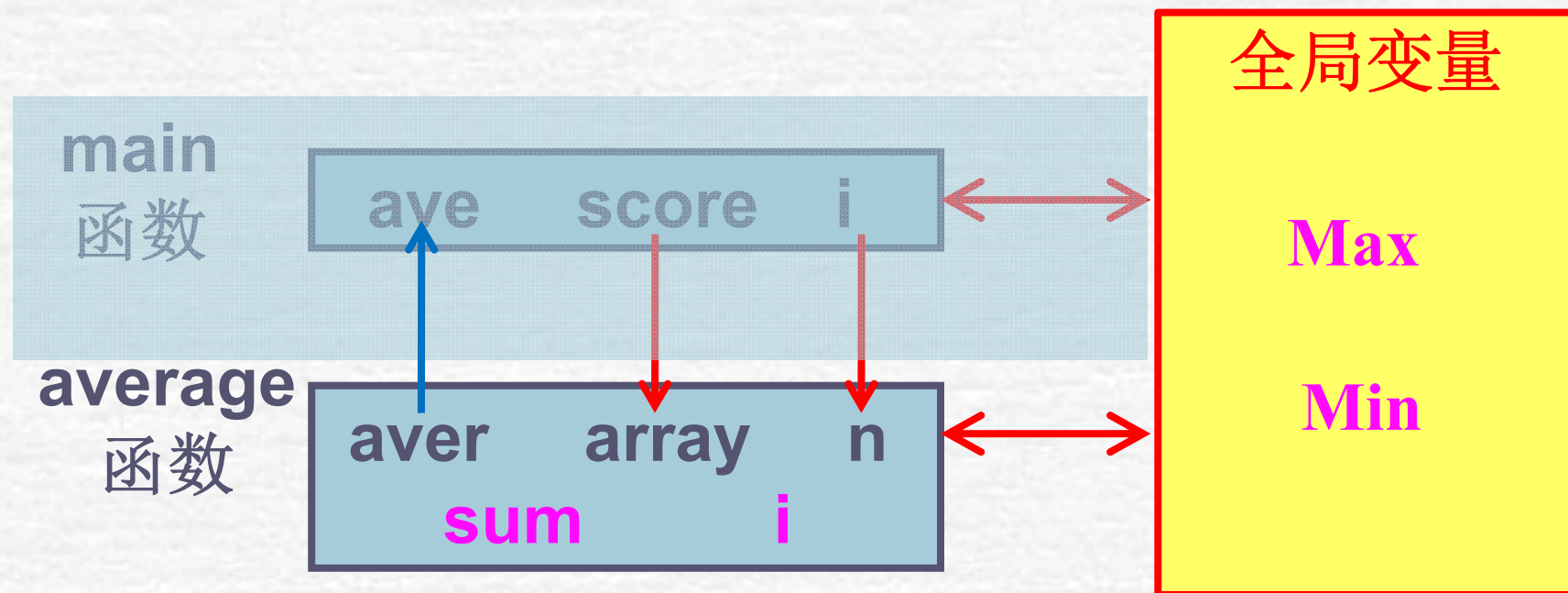
例7.14 有一个一维数组，内放**10**个学生成绩，写一个函数，当主函数调用此函数后，能求出平均分、最高分和最低分。

❏ 解题思路：调用一个函数可以得到一个函数返回值，现在希望通过函数调用能得到**3**个结果。可以利用全局变量来达到此目的。

```
#include <stdio.h>
float Max=0,Min=0;
int main()
{ float average(float array[ ],int n);
  float ave,score[10]; int i;
  printf("Please enter 10 scores:\n");
  for(i=0;i<10;i++)
    scanf("%f",&score[i]);
  ave=average(score,10);
  printf("max=%6.2f\nmin=%6.2f\n
        average=%6.2f\n",Max,Min,ave);
  return 0;
}
```

```
float average(float array[ ],int n)
{ int i; float aver,sum=array[0];
  Max=Min=array[0];
  for(i=1;i<n;i++)
  { if(array[i]>Max) Max=array[i];
    else if(array[i]<Min) Min=array[i];
    sum=sum+array[i];
  }
  aver=sum/n;
  return(aver);
}
```

```
Please enter 10 scores:
89 95 87.5 100 67.5 97 59 84 73 90
max=100.00
min= 59.00
average= 84.20
```

建议不必要时不要使用全局变量！

为了便于区别全局变量和局部变量，在C程序设计时，通常将全局变量名的第一个字母用大写表示。

- 如果在同一个文件中，外部变量与局部变量同名，则在局部变量的作用范围内，外部变量被“屏蔽”，即它不起作用。

就近原则！

例7.15 若外部变量与局部变量同名，分析结果。



```
#include <stdio.h>
```

```
int a=3,b=5;
```

```
int main()
```

```
{ int max(int a,int b);
```

```
    int a=8;
```

```
    printf("max=%d\n",max(a,b));
```

```
    return 0;
```

```
}
```

```
int max(int a,int b)
```

```
{ int c;
```

```
    c=a>b?a:b;
```

```
    return(c);
```

```
}
```

b为全局变量

a为局部变量，仅在此函数内有效




```
#include <stdio.h>
```

```
int a=3,b=5;
```

```
int main()
```

```
{ int max(int a,int b);
```

```
    int a=8;
```

```
    printf("max=%d\n",max(a,b));
```

```
    return 0;
```

```
}
```

```
int max(int a,int b)
```

```
{ int c;
```

```
    c=a>b?a:b; a = 5;
```

```
    return(c);
```

```
}
```

max=8

a、b为局部变量，仅在此函数内有效

【例】

```
int x=0,i=0,j=0;  
void f1(int i,int j)
```

f1中的
局部变量
i, j, x的
作用范围

```
{  
    int x=2;  
    printf("i=%d,j=%d,x=%d\n",i,j,x);  
}
```

```
main()
```

```
{  
    int x=1;  
    printf("i=%d,j=%d,x=%d\n",i,j,x);
```

全局变量
x, i, j的
作用范围

main中
的局部
变量x的
作用范围

```
{  
    int i,j;  
    i=j=5;  
    f1(i++,++j);  
}
```

程序块的
局部变量i, j
的作用范围

```
printf("i=%d,j=%d,x=%d\n",i,j,x);  
}
```